
Algorithm 1 Genetic Algorithm Scheduler

Require: Task set T , core set C , cluster set K , dependency set D

Require: Eligible cores $E_i \subseteq C$ for each task $i \in T$

Require: Task duration d_i , memory demand m_i , minimum start time r_i

Require: Core memory budget B_c^{core} , core WCET scale ets_c

Require: Cluster memory budget $B_k^{cluster}$

Require: Communication penalties and memory penalty weights

Ensure: Best decoded schedule found by the genetic algorithm

- 1: $N \leftarrow |T|$
 - 2: Define chromosome length $2N$ $\triangleright$ (First N genes encode core assignment,
last N genes encode task priority)
 - 3: Define gene space:
 - 4: **for** each task $i \in T$ **do**
 - 5: gene $x_i \in \{0, \dots, |E_i| - 1\}$ $\triangleright$ Index into eligible core list
 - 6: **end for**
 - 7: **for** each task $i \in T$ **do**
 - 8: gene $p_i \in [0, 1)$ $\triangleright$ Priority value
 - 9: **end for**
 - 10: Initialize GA population
 - 11: **while** termination criterion not satisfied **do**
 - 12: **for** each chromosome x in population **do**
 - 13: $F(x) \leftarrow \text{FITNESS}(x)$
 - 14: **end for**
 - 15: Create next generation using GA selection and variation operators
 - 16: **end while**
 - 17: $x^* \leftarrow$ best chromosome found
 - 18: **return** $\text{DECODECHROMOSOME}(x^*)$
-

Algorithm 2 Fitness Evaluation

Require: Chromosome x

Ensure: Fitness value $\phi(x)$

- 1: $(A, P, S, F, C_{\max}, O^{core}, O^{cluster}, Q, J) \leftarrow \text{DECODECHROMOSOME}(x)$
 - 2: $\phi(x) \leftarrow \frac{1}{1 + J}$
 - 3: **return** $\phi(x)$
-

Algorithm 3 Decode Chromosome

Require: Chromosome x of length $2N$ **Ensure:** Assignment, priority order, start times, finish times, max span, core overflow, cluster overflow, and total cost

```
1:  $A \leftarrow \text{DECODEASSIGNMENT}(x)$ 
2:  $P \leftarrow \text{DECODEPRIORITYORDER}(x)$ 
3:  $(S, F) \leftarrow \text{BUILDSCHEDULE}(A, P)$ 
4:  $C_{\max} \leftarrow \max_{i \in T} F_i$ 
5:  $(O^{core}, O^{cluster}) \leftarrow \text{MEMORYOVERFLOW}(A)$ 
6:  $Q \leftarrow \text{COMMUNICATIONCOST}(A)$ 

7:  $J \leftarrow C_{\max}$ 
8:  $J \leftarrow J + \lambda_{core} \sum_{c \in C} O_c^{core}$ 
9:  $J \leftarrow J + \lambda_{cluster} \sum_{k \in K} O_k^{cluster}$ 
10:  $J \leftarrow J + Q$ 
11: return  $(A, P, S, F, C_{\max}, O^{core}, O^{cluster}, Q, J)$ 
```

Algorithm 4 Decode Assignment

Require: Chromosome x **Ensure:** Core assignment A_i for each task $i \in T$

```
1: for each task  $i \in T$  do
2:    $q_i \leftarrow \text{round}(x_i)$ 
3:    $q_i \leftarrow \max(0, \min(q_i, |E_i| - 1))$   $\triangleright$  Clamp to valid eligible-core index
4:    $A_i \leftarrow E_i[q_i]$ 
5: end for
6: return  $A$ 
```

Algorithm 5 Decode Priority Order

Require: Chromosome x **Ensure:** Priority order P

```
1:  $L \leftarrow []$ 
2: for each task  $i \in T$  do
3:    $p_i \leftarrow x_{N+i}$   $\triangleright$  Priority gene for task  $i$ 
4:   Append  $(i, p_i)$  to  $L$ 
5: end for
6: Sort  $L$  by descending  $p_i$ 
7: Break ties by original task index
8:  $P \leftarrow$  ordered task IDs from  $L$ 
9: return  $P$ 
```

Algorithm 6 Build Feasible Schedule from Assignment and Priority Order

Require: Core assignment A , priority order P **Ensure:** Start times S_i and finish times F_i

```
1:  $U \leftarrow T$  ▷ Unscheduled tasks
2:  $S \leftarrow \emptyset, F \leftarrow \emptyset$ 
3: for each core  $c \in C$  do
4:    $a_c \leftarrow 0$  ▷ Earliest time core  $c$  is available
5: end for
6: for each task  $i \in T$  do
7:    $\rho_i \leftarrow$  position of task  $i$  in  $P$  ▷ Lower rank means higher priority
8: end for
9: while  $U \neq \emptyset$  do
10:   $R \leftarrow \{i \in U : \text{pred}(i) \subseteq \text{keys}(F)\}$  ▷ Tasks whose predecessors are
    already scheduled
11:  if  $R = \emptyset$  then
12:    error “No ready task exists” ▷ Dependency graph is infeasible or
    cyclic
13:  end if
14:   $i \leftarrow \arg \min_{j \in R} \rho_j$  ▷ Schedule highest-priority ready tasks
15:   $c \leftarrow A_i$ 
16:  if  $\text{pred}(i) = \emptyset$  then
17:     $f_i^{pred} \leftarrow 0$ 
18:  else
19:     $f_i^{pred} \leftarrow \max_{h \in \text{pred}(i)} F_h$ 
20:  end if
21:   $S_i \leftarrow \max(a_c, f_i^{pred}, r_i)$ 
22:   $F_i \leftarrow S_i + d_i \cdot ets_c$ 
23:   $a_c \leftarrow F_i$ 
24:   $U \leftarrow U \setminus \{i\}$ 
25: end while
26: return  $(S, F)$ 
```

Algorithm 7 Memory Overflow Penalty

Require: Core assignment A **Ensure:** Core overflows O^{core} and cluster overflows $O^{cluster}$

```
1: for each core  $c \in C$  do
2:    $M_c^{core} \leftarrow 0$ 
3: end for
4: for each cluster  $k \in K$  do
5:    $M_k^{cluster} \leftarrow 0$ 
6: end for
7: for each task  $i \in T$  do
8:    $c \leftarrow A_i$ 
9:    $k \leftarrow \text{cluster}(c)$ 
10:   $M_c^{core} \leftarrow M_c^{core} + m_i$ 
11:   $M_k^{cluster} \leftarrow M_k^{cluster} + m_i$ 
12: end for
13: for each core  $c \in C$  do
14:    $O_c^{core} \leftarrow \max(0, M_c^{core} - B_c^{core})$ 
15: end for
16: for each cluster  $k \in K$  do
17:    $O_k^{cluster} \leftarrow \max(0, M_k^{cluster} - B_k^{cluster})$ 
18: end for
19: return  $(O^{core}, O^{cluster})$ 
```

Algorithm 8 Communication Cost

Require: Core assignment A , dependency set D **Ensure:** Communication cost Q

```
1:  $Q \leftarrow 0$ 
2: for each dependency  $(i, j) \in D$  do
3:    $c_i \leftarrow A_i$ 
4:    $c_j \leftarrow A_j$ 
5:   if an explicit communication penalty exists for  $(c_i, c_j)$  then
6:      $Q \leftarrow Q + \text{penalty}(c_i, c_j)$ 
7:   else if  $c_i = c_j$  then
8:      $Q \leftarrow Q + w_{\text{intra\_core}}$ 
9:   else if  $\text{cluster}(c_i) = \text{cluster}(c_j)$  then
10:     $Q \leftarrow Q + w_{\text{inter\_core}}$ 
11:   else
12:     $Q \leftarrow Q + w_{\text{inter\_cluster}}$ 
13:   end if
14: end for
15: return  $Q$ 
```
